

Find a Park Take Home

Candidate brief | 2 hours

The task

Build a small web app that helps residents find local parks. A resident should be able to see parks on a map, browse a list, and open details for a park. Choose a real or fictional municipality.

Save the full, unredacted AI transcript from every tool you use for this exercise, starting with your first prompt. You must submit these logs with your code.

Spend 90 minutes to 2 hours on the whole submission, including testing and your README. Stop at two hours and tell us what you left unfinished. We want a small, polished working app and clear decisions. Extra features are not required.

What to build

- **A map of the parks.** Show where parks are located. Selecting a park should open its details or clearly link to them. The app must work without access to the resident's location.
- **A list of parks.** Residents should also be able to find and select parks without using the map.
- **Park details.** Show the available information, such as the name, address, description, amenities, hours, and images. Handle missing information without crashing or inventing details.
- **An accessible interface.** Make the main tasks work on a phone and desktop. Use readable text and clear labels. Residents should be able to select a park from the list and open its details using a keyboard, with a visible focus indicator. The list and details should provide an alternative to interacting with map markers.

Search, filters, and current-location support are optional. Include them only if the core experience works and you have time. A backend, accounts, and a hosted deployment are not required.

Data and tools

Use the supplied `assets/parks.sample.json`. You may extend it or use another public dataset; describe any changes in your README. The sample includes missing or empty values. Placeholder images are fine.

`assets/parks.sample.json` contains an array of park objects. Each object has this shape:

```
{
  "id": "prospect-park",
  "name": "Prospect Park",
  "description": "A 526-acre park in the heart of Brooklyn...",
  "location": {
    "lat": 40.6602,
    "lng": -73.9690,
    "address": "Brooklyn, NY 11225"
  },
  "amenities": ["playground", "dog-run", "trails", "restrooms", "parking"],
  "hours": "6:00 AM - 1:00 AM",
  "images": ["https://example.com/prospect-1.jpg"],
  "acreage": 526,
  "rating": 4.7
}
```

Fields may be missing or null, and arrays may be empty. Handle those cases gracefully. You may extend the schema; describe any changes in your README. The image URL above is a placeholder.

Choose your own language, framework, and map library. Include any setup needed to run the app. Do not submit private credentials. If a service requires a key, explain how the interviewer can supply it.

Use AI coding tools as you normally would. Using AI is expected. You are responsible for understanding and checking the submitted code, including code generated by a tool.

Submit the full, unredacted AI conversation logs, including your prompts and the tool's responses. Do not edit, shorten, or select excerpts. If your tool cannot export a conversation, contact the recruiter about an alternative.

What to submit

Send a zip file containing:

1. **Source code** for the app.
2. **A README** with run instructions, what works, what you left out, important decisions, known issues, and how you checked the result. Include your approximate time spent and the most important next steps before public use.
3. **Unredacted AI conversation logs** from every tool you used for this exercise.

Follow-up chat

We will spend 30 minutes reviewing your submission together. Have the app and code ready to share. You do not need slides or additional features.

Come prepared to:

- Demo the app and walk through the code.
- Explain your decisions, trade-offs, and what you left out.
- Discuss how you used AI, reviewed its output, and checked the app.
- Explain what you would change before release, including deployment, maintenance, and likely failures.
- Ask questions about the team and role.

Be ready to show a specific piece of code you reviewed closely and explain how you checked it. If you found a mistake, tell us how you found and corrected it. If you did not find one, explain what you checked and what you are still unsure about.

How we evaluate

1. **Usefulness for residents.** Can someone find a park and view its details? Is the app accessible and usable? Did you prioritize the main tasks?
2. **Communication.** Can you explain what you chose, why you chose it, and what you left out in terms your audience can understand?
3. **Code quality and maintainability.** Is the code readable and straightforward to extend? Does it handle missing data and failures sensibly?
4. **AI direction and accountability.** Did you guide the tools, check their output, and take responsibility for the result?
5. **Review and verification.** Can you show how you reviewed the code and checked its behavior, including problems that passing tests might miss?

A modest app that works and is well explained is more useful than an ambitious app with broken core features.

You are not expected to solve every production concern in two hours. Make reasonable assumptions where the prompt is silent and record them in your README. Contact the recruiter if a missing file or setup problem prevents you from starting.